Accelerating Agricultural Software Development: An AI-Agentic Workflow and Practice Based on Vibe Coding

◯Haozhou Wang [1)], Pieter M. Blok [1,2)], Siyao Chen [1)], Wei Guo [1)]

1) Graduate School of Agricultural and Life Science, The University of Tokyo, Tokyo, Japan.

2) Jheronimus Academy of Data Science, Eindhoven University of Technology, ‘s Hertogenbosch, The Netherlands.

Abstract

Although domain experts in agricultural informatics and plant phenotyping frequently develop valuable algorithmic scripts for data processing, the transition of these scripts into industrial-grade Graphical User Interfaces (GUIs) for non-expert users remains an efficiency challenge. Traditional GUI development is often constrained by substantial costs, steep learning curves, and communication barriers between agronomists and software engineers. While recent Large Language Models (LLMs) demonstrate robust coding capabilities for small-scale snippets, the efficient development of large-scale projects remains difficult. This study presents newly formalized AI-agentic skills and workflows based on the vibe coding paradigm. By shifting the role of researchers from manual programmers to system architects, we demonstrate how existing scripts can be rapidly transformed into robust GUI applications. Two case studies, including a 3D point cloud registration tool and a UAV data preprocessing toolkit, illustrate that this workflow reduces development time while preserving code maintainability. This AI-agentic workflow lowers engineering barriers and accelerates the deployment of precision agriculture technologies.

Keyword

AI Agents, Vibe Coding, Software Engineering for Scientists, GUI Development, Agricultural Informatics.

Introduction

The gap between raw research scripts and usable software tools remains a persistent challenge in agricultural informatics and plant phenotyping. Most scripts are hard-coded and executed via command line, making them difficult for non-technical personnel to operate. However, the development of user-friendly GUI software is challenging due to high costs, steep learning curves, and communication barriers between agronomists and software engineers. This delays the widespread deployment of laboratory research findings to practical applications.

Recent developments in large language models show marked progress in coding capabilities. The initial wave of AI-assisted coding relied on web-based chat interfaces. During this chat-based stage, developers manually copied and pasted code, described errors, and explained project structures to the AI. This process proved inefficient and error-prone because the AI lacked access to complete project context, often generating hallucinated code that failed to execute in local environments.

More recent tools gained local file system access, as seen in Claude Code and Gemini CLI. Although these tools reduced manual copying, they introduced new risks. Without proper constraints, autonomous AI agents could cause destructive operations. The text generation mechanisms of LLMs also struggle to maintain coding styles and rules consistent with actual project requirements. To address these issues, researchers began using Global Rules to define technical standards and environment settings. However, even with such rules, maintaining project stability during long-term development remains difficult. To bridge this gap, we present a structured workflow that combines environment isolation, version control, and modular task management, enabling AI to serve as a reliable engineering partner for agricultural software development.

Methods

The tools used in the workflow include Google Antigravity with Claude Opus 4.5 (demo 1) and OpenCode CLI with GPT-

5.3 and the Superpowers plugin (demo 2). These environments provide dual-mode operation: planning mode (or read only mode) generates implementation roadmaps and task lists for human review and comment-based refinement, while fast mode (or execution mode) handles immediate code modifications and execution. This structure enables iterative refinement while maintaining project integrity and supporting visual debugging within integrated development environments.

Risk management relies on strict environmental isolation. Each project operates within local virtual environments (uv or venv), confining all AI operations to specific sandboxes and preventing system-wide dependency conflicts. Git worktree was utilized to create physical separation for experimental features, allowing researchers to safely test AI-generated functionality in parallel directories without destabilizing the main branch or risking data loss.

To maintain code quality, the operating system requirements, coding styles, technology stack, Git commit conventions, and documentation formats were specified within global rules (a markdown file passed to the AI agent during each session). To ensure these rules are enforced as expected, context integrity verification was also implemented by embedding a specific instruction deep within the rules, such as requiring the AI to use a specific greeting or suffix. If the AI stops using this marker, it indicates that the context window has reached capacity and the AI has begun to forget project rules, thereby signaling that a new session is required.

Task decomposition follows the One Session, One Task principle, specifically addressing the Lost in the Middle effect observed when context windows exceed 40-60% capacity (the Lost in the Middle effect). To mitigate this effect, the superpowers plugin was used to follow a strict engineering cycle. In brief, the process begins with the **Brainstorm phase**, which reads background materials and defines logic and requirements through an agent questionnaire. The **Plan phase** then creates a step-by-step roadmap and detailed planning documents in Markdown format, after which the **Execute phase** initiates a new session to read these documents and perform test-driven development (TDD) until all tests passes.

Demonstrations

3DPotatoTwin Registration Tool

We previously released a paired 3D point cloud dataset of potato tubers [1], accompanied by a CLI tool for registration requiring tedious manual parameter adjustment. Using the vibe coding workflow, we developed a functional GUI within one week. This tool provides real-time visual feedback, enabling researchers to efficiently register two point clouds with varying shape and texture differences, guided by a thumbed pin.

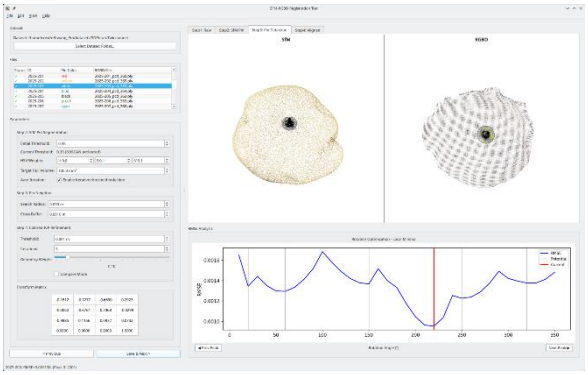

Fig. 1 Interactive interface of the 3DPotatoTwin registration tool, featuring real-time 3D visualization and RMSE optimization feedback for point cloud alignment.

AgriTesseract: Automated UAV Data Preprocessing

We also developed multiple independent scripts for subplot generation, ridge identification, and ID assignment in previous study [2], and recently incorporating annotation-free seedling detection via the SAM3 visual foundation model. However, the lack of a unified interface limited practical usability of these tools. Through the proposed vibe coding workflow, a custom map canvas module with interactive zooming, rotation, and spatial selection capabilities was implemented with minimal effort. We also refactored and integrated legacy code with various environment dependencies into a cohesive project featuring a modern Fluent Design interface. This integration significantly improved efficiency in converting field-scale imagery into structured plant-level data.

Fig. 2 This AgriTesseract GUI screenshot demonstrates the feature of a standalone map canvas for ridge clustering.

References

[1] Wang, H., Blok, P.M., Burridge, J., Jiang, T., Miyauchi, M., Miyamoto, K., Tanaka, K., Guo, W., 2025. 3DPotatoTwin: a paired potato tuber dataset for 3D multi-sensory fusion. Plant Phenomics 7, 100123.

[2] Wang, H., Li, T., Nishida, E., Kato, Y., Fukano, Y., Guo, W., 2023. Drone-Based Harvest Data Prediction Can Reduce On-Farm Food Loss and Improve Farmer Income. Plant Phenomics 5, 0086.